

Can you tell me the status codes starting from 1xx? Only tell me those used in FastAPI commonly.

Here are the **most commonly used HTTP status codes in FastAPI**, organized by category.

1xx – Informational (rarely used in FastAPI APIs)

- 100 Continue
- 101 Switching Protocols (e.g., WebSockets)

2xx – Success

- 200 OK – Request succeeded (default for GET , many PUT requests).
- 201 Created – Resource created successfully (POST).
- 202 Accepted – Request accepted for asynchronous processing.
- 204 No Content – Success, but no response body (often for DELETE).

3xx – Redirection (not commonly returned manually)

- 304 Not Modified – Used with caching.

4xx – Client Errors

- 400 Bad Request – Invalid request.
- 401 Unauthorized – Authentication required or failed.
- 403 Forbidden – Authenticated, but not allowed.
- 404 Not Found – Resource not found.
- 405 Method Not Allowed – Wrong HTTP method used.
- 409 Conflict – Resource conflict (e.g., duplicate entry).
- 422 Unprocessable Entity – Validation error (FastAPI returns this automatically for invalid request data).
- 429 Too Many Requests – Rate limit exceeded.

5xx – Server Errors

- 500 Internal Server Error – Unexpected server error.
- 501 Not Implemented – Feature not implemented.
- 502 Bad Gateway – Invalid response from an upstream server.
- 503 Service Unavailable – Server temporarily unavailable.
- 504 Gateway Timeout – Upstream server timed out.

FastAPI codes you'll use most often

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 422 Unprocessable Entity
- 500 Internal Server Error

These are the status codes you'll encounter and use in the vast majority of FastAPI applications.



1xx – Informational (rarely used in FastAPI APIs)

100 Continue

101 Switching Protocols (e.g., WebSockets) First of all tell me this

The **4xx status codes** mean **the error is on the client's side**. The server understood the request, but it cannot process it because of something wrong with the request.

400 Bad Request

Meaning: The request is malformed or invalid.

Example

Suppose your API expects JSON.

Request

http

POST /patients

Content-Type: application/json

Body:

JSON

```
{  
  "name": "Rahul",
```

The JSON is incomplete (missing `}`), so the server cannot even parse it.

Response

http

400 Bad Request

Use **400** when the request itself is invalid.

401 Unauthorized

Meaning: The user is **not authenticated**.

The server is saying:

"First prove who you are."

Example

Suppose your endpoint requires a JWT token.

Request

http

GET /profile

No token is sent.

Response

http

401 Unauthorized

Or an invalid/expired token is sent.

Again:

http

401 Unauthorized

Remember:

- No login
- Invalid token
- Expired token

403 Forbidden

Meaning: The user is **authenticated**, but **doesn't have permission**.

The server is saying:

"I know who you are, but you are not allowed to do this."

Example

A normal user tries to delete another user's account.

```
DELETE /users/5
```

The user has a valid token.

But only admins can delete users.

Response:

```
http
```

```
403 Forbidden
```

Remember

Authenticated 

Permission 

404 Not Found

Meaning: The requested resource does not exist.

Example

```
GET /patients/999
```

Patient **999** doesn't exist.

Response

http

404 Not Found

FastAPI example:

Python

Run

```
if patient_id not in data:
    raise HTTPException(
        status_code=404,
        detail="Patient not found"
    )
```

This is probably the most common error you'll return manually.

405 Method Not Allowed

Meaning: The URL exists, but that HTTP method is not allowed.

Example

Suppose you only defined:

Python

Run

```
@app.get("/patients")
```

Someone sends

POST /patients

FastAPI automatically returns

http

405 Method Not Allowed

Another example

You only have

DELETE /patients/1

Someone does

```
PUT /patients/1
```

Again

```
http
```

```
405 Method Not Allowed
```

409 Conflict

Meaning: The request conflicts with the current state of the server.

Usually because a resource already exists.

Example

Suppose email must be unique.

Database already contains

```
rahul@gmail.com
```

User sends

```
JSON
```

```
{  
  "email": "rahul@gmail.com"  
}
```

The server responds

```
http
```

```
409 Conflict
```

Another example

Creating a patient whose ID already exists.

422 Unprocessable Entity

This is one of FastAPI's most important status codes.

Meaning: The request format is correct, but the **data fails validation**.

FastAPI automatically returns **422** when a request doesn't match your Pydantic model.

Example

Python

Run

```
class Patient(BaseModel):  
    name: str  
    age: int
```

Client sends

JSON

```
{  
    "name": "Rahul",  
    "age": "twenty"  
}
```

The JSON is valid.

But

age

should be an integer.

FastAPI automatically returns

http

422 Unprocessable Entity

Another example

Missing required field

JSON

```
{  
    "age": 20  
}
```

Again

`http`

`422 Unprocessable Entity`

No need to raise this manually in most cases.

429 Too Many Requests

Meaning: The client is sending too many requests in a short period.

Usually caused by **rate limiting**.

Example

Suppose your API allows

`100 requests per minute`

A user sends

`250 requests`

The server responds

`http`

`429 Too Many Requests`

The client should wait before trying again.

This helps prevent:

- API abuse
- Spam
- Denial-of-service attacks

FastAPI doesn't enforce rate limiting by default, but libraries such as `slowapi` can add it.

Easy way to remember

Status Code

Think of it as...

400

Your request is malformed or invalid.

Status Code	Think of it as...
401	You haven't proved who you are (not authenticated).
403	I know who you are, but you don't have permission.
404	The requested resource doesn't exist.
405	You're using the wrong HTTP method.
409	Your request conflicts with existing data (e.g., duplicate).
422	The request format is fine, but the submitted data doesn't pass validation.
429	You're sending requests too quickly; slow down.

Interview tip: 401 vs 403

This is a very common interview question.

- **401 Unauthorized** → **Authentication problem** (you are not logged in or your token is invalid).
- **403 Forbidden** → **Authorization problem** (you are logged in, but you don't have permission to perform the action).







 Sources

xx – Server Errors

500 Internal Server Error – Unexpected server error.

501 Not Implemented – Feature not implemented.

502 Bad Gateway – Invalid response from an upstream server.

503 Service Unavailable – Server temporarily unavailable.

504 Gateway Timeout – Upstream server timed out. Now explain me these

The **5xx status codes** mean the error is on the server side. The client sent a valid request, but the server couldn't complete it due to an internal problem or an issue with another server.

500 Internal Server Error

Meaning: Something unexpected went wrong on the server.

This is the most common 5xx error.

Example

Suppose your code is:

Python

Run

```
@app.get("/divide")
def divide():
    return 10 / 0
```

This raises a `ZeroDivisionError` .

The client sent a valid request, but your server crashes.

Response

http

```
500 Internal Server Error
```

Another example:

- Database connection crashes unexpectedly.
- Your code has a bug.
- An unhandled exception occurs.

In FastAPI, if you don't catch an unexpected exception, it usually results in **500 Internal Server Error**.

501 Not Implemented

Meaning: The server **does not support** the requested functionality.

The server is saying:

"I understand your request, but I haven't implemented this feature."

Example

Suppose your API documentation says:

POST /payments

supports **Credit Card**, **UPI**, and **PayPal**.

A client sends:

JSON

```
{  
  "payment_method": "Bitcoin"  
}
```

If your server recognizes the request but doesn't support Bitcoin payments yet, it could respond:

http

501 Not Implemented

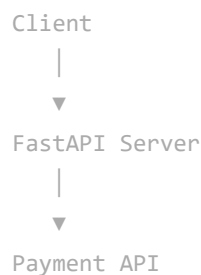
This status is relatively rare in REST APIs. Many APIs instead use **400** or **404**, depending on the situation.

502 Bad Gateway

Meaning: Your server is acting as a **gateway** or **proxy**, and it received an invalid response from another server.

Example

Imagine this architecture:



The client requests payment.

Your FastAPI server forwards the request to the payment service.

The payment service returns an invalid or corrupted response.

Your server responds:

```
http
```

```
502 Bad Gateway
```

The problem is **not your code directly**, but an upstream service.

503 Service Unavailable

Meaning: The server is temporarily unable to handle requests.

Unlike **500**, this is usually a temporary condition.

Example 1: Maintenance

Your server is being updated.

Any request gets:

```
http
```

```
503 Service Unavailable
```

Example 2: Server Overloaded

Suppose your server normally handles:

```
1000 users
```

Suddenly:

```
100,000 users
```

The server cannot process more requests.

It may return:

```
http
```

```
503 Service Unavailable
```

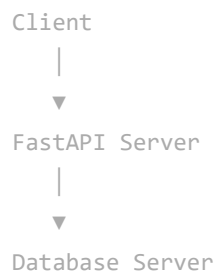
Clients can usually retry later.

504 Gateway Timeout

Meaning: Your server waited too long for another server to respond.

Example

Architecture:



Your FastAPI app asks the database for data.

Normally, the database replies in:

1 second

But today the database is very slow.

After waiting (for example) **30 seconds**, your server gives up.

Response:

http

504 Gateway Timeout

Another example:



The weather API never responds.

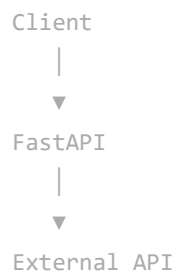
Your server eventually returns:

http

504 Gateway Timeout

502 vs 504 (Easy to remember)

Imagine your FastAPI server calls another API:



- **502 Bad Gateway** → The external API **responded**, but its response was invalid or unexpected.
- **504 Gateway Timeout** → The external API **didn't respond in time**, so your server timed out waiting.

Easy way to remember

Status Code	Meaning
500	My server has an unexpected bug or crashed.
501	This feature isn't implemented on the server.
502	Another server replied with an invalid response.
503	The server is temporarily unavailable or overloaded.
504	Another server took too long to respond.

Which ones do you commonly see in FastAPI?

- **500 Internal Server Error** — Very common during development if an unhandled exception occurs.
- **503 Service Unavailable** — Common in production during maintenance or overload.
- **502 Bad Gateway** and **504 Gateway Timeout** — Common when your FastAPI app is behind a reverse proxy (such as Nginx) or communicates with external services.
- **501 Not Implemented** — Rarely used in typical FastAPI REST APIs.

Now tell me GET,POST,PUT,DELETE,PATCH

These are the **five most important HTTP methods** you'll use in FastAPI and REST APIs.

1. GET

Purpose: Retrieve (read) data.

A **GET** request should **not modify** data on the server.

Example

You want to fetch a patient.

http

```
GET /patients/101
```

Response:

JSON

```
{
  "id": 101,
  "name": "Rahul",
  "age": 25
}
```

FastAPI:

Python

Run

```
@app.get("/patients/{patient_id}")
def get_patient(patient_id: int):
    return data[patient_id]
```

Common status codes

- 200 OK
- 404 Not Found

2. POST

Purpose: Create a new resource.

Every time you send a POST request, you're usually adding something new.

Example

http

POST /patients

Body:

JSON

```
{
  "name": "Rahul",
  "age": 25
}
```

Response:

JSON

```
{
  "id": 101,
  "name": "Rahul",
  "age": 25
}
```

FastAPI:

Python

Run

```
@app.post("/patients", status_code=201)
def create_patient(patient: Patient):
    ...
```

Common status codes

- 201 Created
- 400 Bad Request
- 409 Conflict
- 422 Unprocessable Entity

3. PUT

Purpose: Replace or completely update an existing resource.

Think of **PUT** as replacing the old object with a new one.

Example

Current patient:

JSON

```
{  
  "name": "Rahul",  
  "age": 25,  
  "city": "Patna"  
}
```

PUT request:

http

PUT /patients/101

Body:

JSON

```
{  
  "name": "Ayush",  
  "age": 22,  
  "city": "Delhi"  
}
```

After the update:

JSON

```
{  
  "name": "Ayush",  
  "age": 22,  
  "city": "Delhi"  
}
```

The previous values are replaced.

FastAPI:

```
@app.put("/patients/{patient_id}")
def update_patient(patient_id: int, patient: Patient):
    ...
```

Common status codes

- 200 OK
- 204 No Content
- 404 Not Found
- 422 Unprocessable Entity

4. PATCH

Purpose: Partially update a resource.

Unlike PUT, PATCH updates **only the fields you provide**.

Example

Current patient:

JSON

```
{
    "name": "Rahul",
    "age": 25,
    "city": "Patna"
}
```

PATCH request:

http

PATCH /patients/101

Body:

JSON

```
{
    "age": 26
}
```

After updating:

JSON

```
{  
  "name": "Rahul",  
  "age": 26,  
  "city": "Patna"  
}
```

Only age changed.

FastAPI:

Python

Run

```
@app.patch("/patients/{patient_id}")  
def update_age(patient_id: int, patient: PatientUpdate):  
    ...
```

This is why FastAPI often uses:

Python

Run

```
patient.model_dump(exclude_unset=True)
```

It updates only the fields the client actually sent.

Common status codes

- 200 OK
- 204 No Content
- 404 Not Found
- 422 Unprocessable Entity

5. DELETE

Purpose: Remove a resource.

Example

http

```
DELETE /patients/101
```

Response:

http

204 No Content

The patient is deleted.

FastAPI:

Python

Run

```
@app.delete("/patients/{patient_id}", status_code=204)
def delete_patient(patient_id: int):
    ...
```

Common status codes

- 204 No Content
- 200 OK (if you return a confirmation message)
- 404 Not Found

PUT vs PATCH (Very Important)

Suppose the patient is:

JSON

```
{
  "name": "Rahul",
  "age": 25,
  "city": "Patna"
}
```

Using PUT

Request:

JSON

```
{
  "name": "Ayush",
  "age": 22,
  "city": "Delhi"
}
```

Result:

JSON

```
{  
  "name": "Ayush",  
  "age": 22,  
  "city": "Delhi"  
}
```

The entire resource is replaced.

Using PATCH

Request:

JSON

```
{  
  "age": 22  
}
```

Result:

JSON

```
{  
  "name": "Rahul",  
  "age": 22,  
  "city": "Patna"  
}
```

Only the specified field is updated.

Easy way to remember

Method	Purpose	SQL Analogy
GET	Read data	SELECT
POST	Create new data	INSERT
PUT	Replace an existing record	Full UPDATE (all columns)
PATCH	Partially update a record	Partial UPDATE (specific columns)

Method	Purpose	SQL Analogy
DELETE	Remove data	DELETE

CRUD Mapping

CRUD Operation	HTTP Method
Create	POST
Read	GET
Update (full)	PUT
Update (partial)	PATCH
Delete	DELETE

If you're building a typical FastAPI CRUD application, these five methods form the foundation of almost every REST API you'll create.

       Sources